Basic Assumptions

Rails is an opinionated framework, and this is an opinionated book. Being opinionated means that Rails makes certain tasks easier if you are willing to structure your program the way the Rails core team thinks you should. For this book, being opinionated means not trying to show you every possible way Rails and JavaScript can combine, but instead focusing on the tools I think will be most useful. Perhaps the most important opinion is that we're going to use JavaScript to enhance a mostly server-side Rails application rather than use JavaScript to build a completely separate single-page application (SPA) that only uses Rails as an application programming interface (API).

My basic argument for not writing an SPA is that between Rails and a standard browser, a tremendous amount of complexity is already handled for you. Moving to an SPA structure requires you to build much of that functionality yourself. Over time, the front-end frameworks have gotten better at handling the complexity for you, but to me, it often feels like taking three right turns rather than just taking one left turn. For now and for my money, Rails is less complicated than an SPA for many applications.

That said, there are legitimate places where an SPA might make sense. If your user experience is so different from the normal web structure that the existing behavior of Rails and the browser isn't much help, then an SPA begins to look attractive. If your back end is already an API supporting a mobile app or external services, then an SPA can also act as a consumer of that API, saving you from duplicating view-layer logic (but you can use Rails and web views to go surprisingly far in a mobile app). However, my experience is that most of the time, for most teams, starting by leveraging the Rails view and browser features is the best way to create a great application.

Within that assumption—Rails back end with some front-end interaction—there's still a wide range of tools, architectures, and techniques that might be appropriate for the application. We're going to navigate that space. And within that space, we are going to explore different ways of structuring a